P0847R1
Deducing this

Published Proposal, 2018-10-07

This version:
http://wg21.link/P0847

Authors:
Gašper Ažman (gasper dot azman at gmail dot com)
Simon Brand (simon dot brand at microsoft dot com)
Ben Deane (ben at elbeno dot com)
Barry Revzin (barry dot revzin at gmail dot com)

Audience:
EWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

We propose a new mechanism for specifying or deducing the value category of an instance of a class — in other words, a way to tell from within a member function whether the object it's invoked on is an lvalue or an rvalue; whether it is const or volatile; and the object's type.

Table of Contents

§ 1. Revision History

§ 1.1. Changes since r0

[P0847R0] was presented in Rapperswil in June 2018 using a syntax adjusted from the one used in that paper, using `this Self&& self` to indicate the explicit object parameter rather than the `Self&& this self` that appeared in r0 of our paper.

EWG strongly encouraged us to look in two new directions:

- a different syntax, placing the object parameter's type after the member function's parameter declarations (where the *cv-ref* qualifiers are today)
- a different name lookup scheme, which could prevent implicit/unqualified access from within new-style member functions that have an explicit self-type annotation, regardless of syntax.

This revision carefully explores both of these directions, presents different syntaxes and lookup schemes, and discusses in depth multiple use cases and how each syntax can or cannot address them.

§ 2. Motivation

In C++03, member functions could have *cv*-qualifications, so it was possible to have scenarios where a particular class would want both a `const` and non-`const` overload of a particular member. (Note that it was also possible to want `volatile` overloads, but those are less common and thus are not examined here.) In these cases, both overloads do the same thing — the only difference is in the types being accessed and used. This was handled by either duplicating the function while adjusting types and qualifications as necessary, or having one overload delegate to the other. An example of the latter can be found in Scott Meyers's "Effective C++" [Effective], Item 3:

```
class TextBlock {
public:
  char const& operator[](size_t position) const {
    // ...
    return text[position];
  }

  char& operator[](size_t position) {
    return const_cast<char&>(
      static_cast<TextBlock const&>(*this)[position]
    );
  }
  // ...
};
```

Arguably, neither duplication nor delegation via `const_cast` are great solutions, but they work.

In C++11, member functions acquired a new axis to specialize on: ref-qualifiers. Now, instead of potentially needing two overloads of a single member function, we might need four: `&`, `const&`, `&&`, or `const&&`. We have three approaches to deal with this:

- We implement the same member four times;
- We have three overloads delegate to the fourth; or
- We have all four overloads delegate to a helper in the form of a private static member function.

One example of the latter might be the overload set for `optional<T>::value()`, implemented as:

Quadruplication

```
template <typename T>
class optional {
  // ...
  constexpr T& value() & {
    if (has_value()) {
      return this->m_value;
    }
    throw bad_optional_access();
  }

  constexpr T const& value() const& {
    if (has_value()) {
      return this->m_value;
    }
    throw bad_optional_access();
  }

  constexpr T&& value() && {
    if (has_value()) {
      return move(this->m_value);
    }
    throw bad_optional_access();
  }

  constexpr T const&&
  value() const&& {
    if (has_value()) {
      return move(this->m_value);
    }
    throw bad_optional_access();
  }
  // ...
};
```

Delegation to 4th

```
template <typename T>
class optional {
  // ...
  constexpr T& value() & {
    return const_cast<T&>(
      static_cast<optional const&>(
        *this).value());
  }

  constexpr T const& value() const& {
    if (has_value()) {
      return this->m_value;
    }
    throw bad_optional_access();
  }

  constexpr T&& value() && {
    return const_cast<T&&>(
      static_cast<optional const&>(
        *this).value());
  }

  constexpr T const&&
  value() const&& {
    return static_cast<T const&&>(
      value());
  }
  // ...
};
```

Delegation t

```
template <typename T
class optional {
  // ...
  constexpr T& value
    return value_imp
  }

  constexpr T const&
    return value_imp
  }

  constexpr T&& valu
    return value_imp
  }

  constexpr T const&
  value() const&& {
    return value_imp
  }

private:
  template <typename
  static decltype(au
  value_impl(Opt&& o
    if (!opt.has_val
      throw bad_opti
    }
    return forward<O
  }
  // ...
};
```

This is far from a complicated function, but essentially repeating the same code four times — or using artificial delegation to avoid doing so — begs a rewrite. Unfortunately, it's impossible to improve; we *must* implement it this way. It seems we should be able to abstract away the qualifiers as we can for non-member functions, where we simply don't have this problem:

```
template <typename T>
class optional {
    // ...
    template <typename Opt>
    friend decltype(auto) value(Opt&& o) {
        if (o.has_value()) {
            return forward<Opt>(o).m_value;
        }
        throw bad_optional_access();
    }
    // ...
};
```

All four cases are now handled with just one function... except it's a non-member function, not a member function. Different semantics, different syntax, doesn't help.

There are many cases where we need two or four overloads of the same member function for different `const`- or ref-qualifiers. More than that, there are likely additional cases where a class should have four overloads of a particular member function but, due to developer laziness, doesn't. We think that there are enough such cases to merit a better solution than simply "write it, write it again, then write it two more times."

§ 3. Proposal

We propose a new way of declaring non-static member functions that will allow for deducing the type and value category of the class instance parameter while still being invocable with regular member function syntax.

We believe that the ability to write *cv-ref qualifier*-aware member function templates without duplication will improve code maintainability, decrease the likelihood of bugs, and make fast, correct code easier to write.

The proposal is sufficiently general and orthogonal to allow for several new exciting features and design patterns for C++:

- recursive lambdas
- a new approach to mixins, a CRTP without the CRT
- move-or-copy-into-parameter support for member functions
- efficiency by avoiding double indirection with invocation
- perfect, sfinae-friendly call wrappers

These are explored in detail in the examples section.

This proposal assumes the existence of two library additions, though it does not propose them:

- `like_t`, a metafunction that applies the *cv*- and *ref*-qualifiers of the first type onto the second (e.g. `like_t<int&, double>` is `double&`, `like_t<X const&&, Y>` is `Y const&&`, etc.)
- `forward_like`, a version of `forward` that is intended to forward a variable not based on its own type but instead based on some other type. `forward_like<T>(u)` is short-hand for `forward<like_t<T,decltype(u)>>(u)`.

§ 3.1. Proposed Syntaxes

There are four syntax options for solving this problem which will be used throughout the examples in the rest of our proposal; this section briefly introduces those options. Semantics are more thoroughly explained in later sections. The various syntaxes used imply subtly different semantics which are called out where relevant. This paper takes the position that only one option be chosen, depending on the desired characteristics.

3.1.1. Explicit `this`-annotated parameter

A non-static member function can be declared to take as its first parameter an *explicit object parameter*, denoted with the prefixed keyword `this`. Once we elevate the object parameter to a proper function parameter, it can be deduced following normal function template deduction rules:

```
struct X {
    void foo(this X const& self, int i);

    template <typename Self>
    void bar(this Self&& self);
};

struct D : X { };

void ex(X& x, D const& d) {
    x.foo(42);      // 'self' is bound to 'x', 'i' is 42
    x.bar();        // deduces Self as X&, calls X::bar<X&>
    move(x).bar();  // deduces Self as X, calls X::bar<X>

    d.foo(17);      // 'self' is bound to 'd'
    d.bar();        // deduces Self as D const&, calls X::bar<D const&>
}
```

Member functions with an explicit object parameter cannot be `static` or have *cv*- or *ref*-qualifiers.

A call to a member function will interpret the object argument as the first (`this`-annotated) parameter to it; the first argument in the parenthesized expression list is then interpreted as the second parameter, and so forth.

Following normal deduction rules, the template parameter corresponding to the explicit object parameter can deduce to a type derived from the class in which the member function is declared, as in the example above for `d.bar()`).

3.1.1.1. Lambda version

The lambda version of the above, for reference:

```
vector captured = {1, 2, 3, 4};
[captured](this auto&& self) -> decltype(auto) {
  return forward_like<decltype(self)>(captured);
}

[captured]<class Self>(this Self&& self) -> decltype(auto) {
  return forward_like<Self>(captured);
}
```

The lambdas can either move or copy from the capture, depending on whether the lambda is an lvalue or an rvalue.

3.1.2. Explicit `this` parameter

A non-static member function can be declared to take `this` as its first parameter. As with §3.1.1 Explicit this-annotated parameter, once we elevate the object parameter to a proper function parameter, it can be deduced following normal function template deduction rules. In this case, instead of remaining a pointer, `this` becomes a reference of the type the parameter suggested by the parameter, fixing a long-standing oversight of the language, stemming from before the invention of references.

```
struct X {
    void foo(X const& this, int i);

    template <typename Self>
    void bar(Self&& this);
};

struct D : X { };

void ex(X& x, D const& d) {
    x.foo(42);      // 'this' is bound to 'x', 'i' is 42
    x.bar();        // deduces Self as X&, calls X::bar<X&>
    move(x).bar();  // deduces Self as X, calls X::bar<X>

    d.foo(17);      // 'this' is bound to 'd'
    d.bar();        // deduces Self as D const&, calls X::bar<D const&>
}
```

As with §3.1.1 Explicit this-annotated parameter, member functions with an explicit object parameter cannot be `static` or have *cv*- or *ref*-qualifiers.

A call to a member function will interpret the object argument as its first `this` parameter and the first argument in the parenthesized expression list as its second parameter (and so on).

Following normal deduction rules, the template parameter corresponding to the explicit object parameter can deduce to a type derived from the class in which the member function is declared, as in the example above for `d.bar()`.

§ *3.1.2.1. Lambda version*

The lambda version of the above, for reference:

```
vector captured = {1, 2, 3, 4};
[captured](auto&& this) -> decltype(auto) {
  return forward_like<decltype(this)>(captured);
}

[captured]<class Self>(Self&& this) -> decltype(auto) {
  return forward_like<Self>(captured);
}
```

§ 3.1.3. Trailing type with identifier

In C++17, member functions can optionally have a *cv-qualifier* and a *ref-qualifier*. This can be extended to allow a full type and an identifier so that the trailer of the member function declaration effectively becomes a variable declaration for the object parameter:

```
struct X {
    void foo(int i) X const& self;

    template <typename Self>
    void bar() Self&& self;
};

struct D : X { };

void ex(X& x, D const& d) {
    x.foo(42);      // 'self' is bound to 'x', 'i' is 42
    x.bar();        // deduces Self as X&, calls X::bar<X&>
    move(x).bar();  // deduces Self as X, calls X::bar<X>

    d.foo(17);      // 'self' is bound to 'd'
    d.bar();        // deduces Self as D const&, calls X::bar<D const&>
}
```

This takes the previously introduced §3.1.1 Explicit this-annotated parameter and simply declares it after the parameter declaration clause instead of at the front of it. The grammar is extended with a parameter declaration, where current-syntax member functions behave as if the class name were omitted.

Member functions with an trailing object type cannot be `static`.

3.1.3.1. Lambda version

The lambda version of the above, for reference:

```
vector captured = {1, 2, 3, 4};
[captured]<class Self>() Self&& self -> decltype(auto) {
  return forward_like<Self>(captured);
}
```

Note that this syntax obviates `mutable` and `const` qualifiers for the lambda and introduces significant parsing issues due to `self` being both completely freeform and optional.

3.1.4. Trailing type sans identifier

Similar to the above, except without a possible identifier.

```
struct X {
    void foo(int i) X const&;

    template <typename Self>
    void bar() Self&&;
};

struct D : X { };

void ex(X& x, D const& d) {
    x.foo(42);      // this is a X const*
    x.bar();        // deduces Self as X&, this is a X*
    move(x).bar();  // deduces Self as X, this is a X*

    d.foo(17);      // this is a X const*
    d.bar();        // deduces Self as D const&, this is a D const*
}
```

Because we need a way to refer to the derived object but are no longer introducing an identifier for it, we are forced to use `this`. The optionality of introducing a type-identifier would suggest that `this` not change meaning in the body of such a member function; in other words, `this` stays a pointer to an appropriately cv-ref qualified object.

Refer to §3.2.4 Name lookup: within member functions for a discussion of the various options for handling name lookup in the member function bodies for each syntax.

3.1.4.1. Lambda version

The lambda version of the above, for reference:

```
vector captured = {1, 2, 3, 4};
[captured]<class Self>() Self&& -> decltype(auto) {
  return forward_like<Self>(captured);
}
```

One should note that this syntax obviates `mutable` and `const` qualifiers for the lambda.

3.1.5. Quick comparison

This serves as a brief demonstration of how to write `optional::value()` and `optional::operator->()` in just two functions instead of six with no duplication using each of the four proposed syntaxes.

There are several options as to the semantics of `this` within member function bodies, with this quick illustrative example being the most conservative approach:

§3.1.1 Explicit this-annotated parameter

```
template <typename T>
struct optional {
  template <typename Self>
  constexpr auto&& value(this Self&& self) {
    if (!self.has_value()) {
      throw bad_optional_access();
    }

    return forward<Self>(self).m_value;
  }

  template <typename Self>
  constexpr auto operator->(this Self&& self) {
    return addressof(self.m_value);
  }
};
```

§3.1.2 Explicit this parameter

```
template <typename T>
struct optional {
  template <typename Self>
  constexpr auto&& value(Self&& this) {
    if (!this.has_value()) {
      throw bad_optional_access();
    }

    return forward<Self>(this).m_value;
  }

  template <typename Self>
  constexpr auto operator->(Self&& this) {
    return addressof(this.m_value);
  }
};
```

§3.1.3 Trailing type with identifier

§3.1.4 Trailing type sans identifier

```cpp
template <typename T>
struct optional {
  template <typename Self>
  constexpr auto&& value() Self&& self {
    if (!self.has_value()) {
      throw bad_optional_access();
    }

    return forward<Self>(self).m_value;
  }

  template <typename Self>
  constexpr auto operator->() Self self {
    return addressof(self.m_value);
  }
};
```

```cpp
template <typename T>
struct optional {
  template <typename Self>
  constexpr auto&& value() Self&& {
    if (!this->has_value()) {
      throw bad_optional_access();
    }

    return forward<Self>(*this).m_value;
  }

  template <typename Self>
  constexpr auto operator->() Self {
    return addressof(this->m_value);
  }
};
```

Note that the two syntaxes with trailing object types declare the object parameter to have only type `Self`, while the explicit object parameter syntaxes still use `Self&&`. This is explained further when we discuss taking `this` by value.

§ 3.2. Proposed semantics

What follows is a description of how deducing `this` affects all important language constructs — name lookup, type deduction, overload resolution, and so forth.

This is a strict extension to the language. Depending on the syntax chosen, either all or nearly all existing syntax remains valid (see §3.2.10 Parsing issues for a pathological case that may change some meaning should §3.1.3 Trailing type with identifier or §3.1.4 Trailing type sans identifier be chosen).

§ 3.2.1. Name lookup: candidate functions

In C++17, name lookup includes both static and non-static member functions found by regular class lookup when invoking a named function or an operator, including the call operator, on an object of class type. Non-static member functions are treated as if there were an implicit object parameter whose type is an lvalue or rvalue reference to *cv* `X` (where the reference and *cv* qualifiers are determined based on the function's own qualifiers) which binds to the object on which the function was invoked.

For non-static member functions using **any of the new syntaxes** (whether an **explicit** object parameter or a full **explicit** trailing object type), lookup will work the same way as other member functions in C++17, with one exception: rather than implicitly determining the type of the object parameter based on the *cv*- and *ref*-qualifiers of the member function, these are now explicitly determined by the provided type of the explicit object parameter. The following examples illustrate this concept, with the two other syntaxes omitted for brevity, since they behave identically.

C++17	§3.1.1 Explicit this-annotated parameter	§3.1.4 Trailing t

```cpp
struct X {
  // implicit object has type X&
  void foo() &;

  // implicit object has type X const&
  void foo() const&;

  // implicit object has type X&&
  void bar() &&;
};
```

```cpp
struct X {
  // explicit object has type X&
  void foo(this X&);

  // explicit object has type X const&
  void foo(this X const&);

  // explicit object has type X&&
  void bar(this X&&);
};
```

```cpp
struct X {
  // explicit obje
  void foo() X&;

  // explicit obje
  void foo() X con

  // explicit obje
  void bar() X&&;
};
```

Name lookup on an expression like `obj.foo()` in C++17 would find both overloads of `foo` in the first column, with the non-const overload discarded should `obj` be const.

With any of the proposed syntaxes, `obj.foo()` would continue to find both overloads of `foo`, with identical behaviour to C++17.

The only change in how we look up candidate functions is in the case of an explicit object parameter, where the argument list is shifted by one. The first listed parameter is bound to the object argument, and the second listed parameter corresponds to the first argument of the call expression.

This paper does not propose any changes to overload *resolution* but merely suggests extending the candidate set to include non-static member functions and member function templates written in a new syntax. Therefore, given a call to `x.foo()`, overload resolution would still select the first `foo()` overload if `x` is not `const` and the second if it is.

The behaviors of the three columns are exactly equivalent as proposed.

The only change as far as candidates are concerned is that the proposal allows for deduction of the object parameter, which is new for the language.

§ 3.2.2. Type deduction

One of the main motivations of this proposal is to deduce the *cv*-qualifiers and value category of the class object, which requires that the explicit member object or type be deducible from the object on which the member function is invoked.

If the type of the object parameter is a template parameter, all of the usual template deduction rules apply as expected:

§3.1.1 Explicit this-annotated parameter

```
struct X {
  template <typename Self>
  void foo(this Self&&, int);
};

struct D : X { };

void ex(X& x, D& d) {
    x.foo(1);       // Self=X&
    move(x).foo(2); // Self=X
    d.foo(3);       // Self=D&
}
```

§3.1.4 Trailing type sans identifier

```
struct X {
  template <typename Self>
  void foo(int) Self&&;
};

struct D : X { };

void ex(X& x, D& d) {
    x.foo(1);       // Self=X&
    move(x).foo(2); // Self=X
    d.foo(3);       // Self=D&
}
```

It's important to stress that deduction is able to deduce a derived type, which is extremely powerful. In the last line, regardless of syntax, `Self` deduces as `D&`. This has implications for §3.2.4 Name lookup: within member functions, and leads to a potential template deduction extension.

§ 3.2.3. By value `this`

But what if the explicit type does not have reference type? This is one key point where the proposed rules for having an explicit object parameter and having an trailing object type diverge. For both syntaxes, there is a clear meaning — it's just that it is very different clear meaning between the two.

§ *3.2.3.1. By value `this` in explicit syntaxes*

In the case of §3.1.1 Explicit this-annotated parameter and §3.1.2 Explicit this parameter, what should this mean:

```
struct less_than {
    template <typename T, typename U>
    bool operator()(this less_than, T const& lhs, U const& rhs) {
        return lhs < rhs;
    }
};

less_than{}(4, 5);
```

Clearly, the parameter specification should not lie, and the first parameter (`less_than{}`) is passed by value.

Following the proposed rules for candidate lookup, the call operator here would be a candidate, with the object parameter binding to the (empty) object and the other two parameters binding to the arguments. Having a value parameter is nothing new in the language at all — it has a clear and obvious meaning, but we've never been able to take an object parameter by value before. For cases in which this might be desirable, see §4.4 By-value member functions.

However, with a by-value explicit object parameter, we still must answer the question of what `this` refers to within the function body. Regardless of the choice we make for §3.2.4 Name lookup: within member functions, there is only one meaningful semantic choice here: there is no `this`. In the above example, `less_than` doesn't *refer* to the object parameter, it *is* the only object parameter. Declaring a function in this way is equivalent to declaring a non-member `friend` function, except that we effectively opt-in to the usual member function call syntax. This also determines its pointer type — it is a free function type.

§ *3.2.3.2. By value `this` in trailing syntaxes*

In the case of trailing object type, what should this mean:

```
template <typename T, size_t N>
struct array {
    template <typename Self>
    auto& operator[](size_t) Self;
};
```

Clearly, since the object type is an *optional* part of the syntax, omitting it should produce the same results as it would currently.

A common source of member function duplication revolves around wanting non-`const` and `const` overloads of a member function that otherwise do exactly the same thing. The normal template deduction rules would drop *cv*-qualifiers, meaning:

```
using A = array<int, 10>;
void ex(A& a, A const&, ca) {
    a[0];  // deduces Self=A
    ca[0]; // deduces Self=A, same function
}
```

But with normal member function declarations today, we don't have the same notion of a "value." The implicit object parameter is always a reference. We're used to writing either nothing or `const` at the end of member functions, which is analogous to writing `C` or `C const`. It would follow that allowing the deduction of a naked (i.e. non-reference) template parameter to preserve *cv*-qualifiers would produce the expected behavior and be quite useful.

Should the trailing syntax be chosen, we propose that `ca[0]` deduces `Self` as `A const` in the previous example.

This mimics today's behavior where a trailing `const` qualifier does not mean `const&` — it only means `const`. Without such a change to template deduction, `Self` would always deduce as `A`, and hence be pointless; `Self&` would deduce as `A&` or `A const&` but not allow binding to rvalues; and `Self&&` would give us different functions for lvalues and rvalues, which is unnecessary and leads to code bloat.

§ 3.2.4. Name lookup: within member functions

So far, we've only considered how member functions with trailing object types are found with name lookup and how they deduce that parameter. Now we move on to how the bodies of these functions actually behave.

Since either the explicit object parameter or trailing object type is deduced from the object on which the function is called, this has the possible effect of deducing *derived* types. We must carefully consider how name lookup works in this context.

To avoid repetition, we'll use an explicit object parameter for this example. Other syntaxes have identical or a subset of these possibilities, modulo some renaming:

```
struct B {
    int i = 0;

    template <typename Self> auto&& f1(this Self&&) { return i;  }
    template <typename Self> auto&& f2(this Self&&) { return this->i; }
    template <typename Self> auto&& f3(this Self&&) { return forward_like<Self>(*this).i; }
    template <typename Self> auto&& f4(this Self&&) { return forward<Self>(*this).i; }
    template <typename Self> auto&& f5(this Self&& self) { return forward<Self>(self).i; }
};

struct D : B {
    // shadows B::i
    double i = 3.14;
};
```

The question is, what do each of these five functions do? Should any of them be ill-formed? What is the safest option?

We believe that there are three approaches to choose from:

1. If there is an explicit object parameter, `this` is inaccessible, and each access must be through `self`. There is no implicit lookup of members through `this`. This makes `f1` through `f4` ill-formed and only `f5` well-formed. However, while `B().f5()` returns a reference to `B::i`, `D().f5()` returns a reference to `D::i`, since `self` is a reference to `D`.
2. If there is an explicit object parameter, `this` is accessible and points to the base subobject. There is no implicit lookup of members; all access must be through `this` or `self` explicitly. This makes `f1` ill-formed. `f2` would be well-formed and always return a reference to `B::i`. Most importantly, `this` would be *dependent* if the explicit object parameter was deduced. `this->i` is always going to be an `int` but it could be either an `int` or an `int const` depending on whether the `B` object is const. `f3` would always be well-formed and would be the correct way to return a forwarding reference to `B::i`. `f4` would be well-formed when invoked on `B` but ill-formed if invoked on `D` because of the requested implicit downcast. As before, `f5` would be well-formed.
3. `this` is always accessible and points to the base subobject; we allow implicit lookup as in C++17. This is mostly the same as the previous choice, except that now `f1` is well-formed and exactly equivalent to `f2`.

Based on these three name lookup semantics, here is a comparison of the implementation of `optional::value()` that takes great care to avoid accessing anything from the derived object:

`this` inaccessible	

```
template <typename Self>
auto&& value(this Self&& self) {
  if (!self.optional::has_value()) {
    throw bad_optional_access();
  }

  return forward<Self>(self).optional::m_value;
}
```

`this` accessible but explicit	`template <typename Self>`
	`auto&& value(this Self&&) {`
	`  if (!this->has_value()) {`
	`  throw bad_optional_access();`
	`  }`
	`  return forward_like<Self>(*this).m_value;`
	`}`

`this` accessible and implicit	`template <typename Self>`
	`auto&& value(this Self&&) {`
	`  if (!has_value()) {`
	`  throw bad_optional_access();`
	`  }`
	`  return forward_like<Self>(m_value);`
	`}`

Note that in the latter two choices, we do not even provide a name to the explicit object parameter, since it is not needed, although we could have chosen to do so anyway.

What do these options mean when considering the trailing type syntax? If we allow an *identifier*, then the name lookup semantics would be exactly the same as having an explicit object parameter — the named parameter is in a different location in the declaration, but otherwise has the same meaning.

If we take the trailing object type syntax *without an identifier*, we can't differentiate between what `this` might refer to and what `self` might refer to. We can really only have one possible name: `this`. And `this` would now have to change types, becoming a `const` pointer to `remove_reference_t<T>`.

The same is true of the §3.1.2 Explicit this parameter syntax: `this` is the only name that exists and refers to the most derived object, so implicit access is disallowed.

```
struct X {
    void a();            // this is a X* const
    void b() const;      // this is a X const* const
    void c() &&;         // this is a X* const (ref-qualifiers don't count)

    void d() X;          // this is a X* const (same as a)
    void e() X&&;        // this is a X* const (same as c)

    template <typename S>
    void f() S;          // this is a S* const (which can be a pointer to const or not!)
    template <typename S>
    void g() S&;         // this is a S* const
    template <typename S>
    void h() S&&;        // this is a remove_reference_t<S>* const
};
```

For some of these member functions, `this` might not point to `X`. It might point to a type derived from `X`. With the trailing type syntax, we end up with a slightly different choice of syntaxes:

```
struct B {
    int i = 0;

    template <typename Self> auto&& f1() Self&& { return i;  }
    template <typename Self> auto&& f2() Self&& { return this->i; }
    template <typename Self> auto&& f3() Self&& { return forward_like<Self>(*this).i; }
    template <typename Self> auto&& f4() Self&& { return forward<Self>(*this).i; }
};
```

First, note that `f3` and `f4` are equivalent with this syntax. Since `this` is a pointer to `Self`, there is no differentiation. Our only real options are:

1. `this` is not implicitly accessible and you need to qualify every access — that is, `f1` is ill-formed.
2. `this` is implicitly accessible, but since `this` is no longer necessarily a pointer to `B`, `f1` does not necessary return `B::i`.

Without an identifier, there is no way to syntactically differentiate between *specifically* accessing something in `B` and accessing something through whatever is deduced as the object parameter. The same is true if, when having an identifier, we choose option #1 (`this` is always inaccessible). We would have to write one of the following:

```
// for each of these examples, we are using 'self' as the named
// identifier for the object parameter. If we are using the syntax
// that does not allow for an identifier, replace it mentally with
// *this instead.

// explicitly cast self to the appropriately qualified B
// note that we have to cast self, not self.i
return static_cast<like_t<Self, B>&&>(self).i;

// use the explicit subobject syntax. Note that this is always
// an lvalue reference - not a forwarding reference
return self.B::i;

// use the explicit subobject syntax to get a forwarding reference
return forward<Self>(self).B::i;
```

This is quite complex, so we believe there to be a benefit in having the ability to syntactically differentiate between referring to the deduced most-derived object and the non-deduced class-we're-in object.

§ 3.2.5. Writing the function pointer types for such functions

The proposed change allows us to deduce the object parameter's value category and *cv*-qualifiers, but the member functions themselves are otherwise the same as today, with no change to their types.

In other words, given:

```
struct Y {
    int f(int, int) const&;
    int g(int, int) Y const&;
    int h(this Y const&, int, int);
};
```

`Y::f`, `Y::g`, and `Y::h` are equivalent from a signature standpoint, so all of them have type `int(Y::*)(int, int) const&`.

This becomes especially interesting when deduction kicks in. These rules are the same regardless of syntax, so to avoid repetition, we'll use the explicit object parameter syntax:

```
struct B {
    template <typename Self>
    void foo(this Self&&);
};

struct D : B { };
```

The type of `&B::foo<B>` is `void (B::*)() &&` and the type of `&B::foo<B const&>` is `void (B::*)() const&`. This is just a normal member function. The type of `&D::foo<B>` is `void (B::*)() &&`. This is effectively the same thing that would happen if `foo` were a normal C++17 member function. The type of `&B::foo<D>` is `void (D::*)() &&`. That is, it behaves as if it were a member function of `D`.

By-value object parameters can be allowed for either §3.1.1 Explicit this-annotated parameter or §3.1.2 Explicit this parameter. Taking the address of these functions does not give you a pointer to member function, but a pointer to function, following their semantics as being effectively non-member `friend` functions:

```
template <typename T>
struct less_than {
    bool operator()(this less_than, T const&, T const&);
};
```

The type of `&less_than<int>::operator()` is `bool(*)(less_than<int>, int const&, int const&)` and follows the usual rules of invocation:

```
less_than<int> lt;
auto p = &less_than<int>::operator();

lt(1, 2);            // ok
p(lt, 1, 2);         // ok
(lt.*p)(1, 2);       // error: p is not a pointer to member function
invoke(p, lt, 1, 2); // ok
```

§ 3.2.6. Pathological cases

It is important to mention the pathological cases. First, what happens if `D` is incomplete but becomes valid later?

```
struct D;
struct B {
    void foo(this D&);
};
struct D : B { };
```

Following the precedent of [P0929R2], we think this should be fine, albeit strange. If `D` is incomplete, we simply postpone checking until the point of call or formation of pointer to member, etc. At that point, the call will either not be viable or the formation of pointer-to-member would be ill-formed.

For unrelated complete classes or non-classes:

```
struct A { };
struct B {
    void foo(this A&);
    void bar(this int);
};
```

The declaration can be immediately diagnosed as ill-formed.

Another interesting case, courtesy of Jens Maurer:

```
struct D;
struct B {
  int f1(this D);
};
struct D1 : B { };
struct D2 : B { };
struct D : D1, D2 { };

int x = D().f1();  // error: ambiguous lookup
int y = B().f1();  // error: B is not implicitly convertible to D
auto z = &B::f1;   // ok
z(D());            // ok
```

Even though both `D().f1()` and `B().f1()` are ill-formed, for entirely different reasons, taking a pointer to `&B::f1` is acceptable — its type is `int(*)(D)` — and that function pointer can be invoked with a `D`. Actually invoking this function does not require any further name lookup or conversion because by-value member functions do not have an implicit object parameter in this syntax (see §3.2.3 By value this).

3.2.7. Teachability Implications

Explicitly naming the object as the `this`-designated first parameter fits within many programmers' mental models of the `this` pointer being the first parameter to member functions "under the hood" and is comparable to its usage in other languages, e.g. Python and Rust. It also works as a more obvious way to teach how `std::bind`, `std::thread`, `std::function`, and others work with a member function pointer by making the pointer explicit.

A natural extension of having trailing *cv*- and *ref-qualifiers* to non-static member functions is providing an explicit type to which those qualifiers refer in place of the implied class type. This keeps all of the qualifiers together, which is more idiomatic C++ in this sense. The ability to deduce this type follows once we have somewhere we can name it.

We do not believe there to be any teachability problems with either choice of syntax.

3.2.8. Can `static` member functions have an explicit object type?

No. Static member functions currently do not have an implicit object parameter, and therefore have no reason to provide an explicit one.

3.2.9. Interplays with capturing `[this]` and `[*this]` in lambdas

Impacts differ depending on the chosen syntax.

3.2.9.1. Syntaxes with an identifier

For §3.1.1 Explicit this-annotated parameter and §3.1.3 Trailing type with identifier, interoperability is perfect, since they do not impact the meaning of `this` in a function body. The introduced identifier `self` can then be used to refer to the lambda instance from the body.

3.2.9.2. Syntaxes without an identifier

For §3.1.2 Explicit this parameter and §3.1.4 Trailing type sans identifier, things get more complex, since they use `this` for their own purposes.

The sensible meaning differs between §3.1.2 Explicit this parameter and §3.1.4 Trailing type sans identifier, since they introduce their parameters in different ways.

With §3.1.2 Explicit this parameter, `this` is a parameter of the implied type. Parameters shadow closure members, so `this` should as well, making the capture of `this` in lambdas *a lot* less powerful. One possible solution is to allow explicit referencing of closure members in this case so that `this.this` becomes the captured `this` object.

With §3.1.4 Trailing type sans identifier, behavior must stay the same, meaning that there is no possible way to refer to the lambda object. This effectively prohibits §3.1.4 Trailing type sans identifier from solving the recursive lambda problem. The rules for what `this` means in a lambda today still apply: `this` can only ever refer to a captured member pointer of an outer member function, and can never be a pointer to the lambda instance itself.

```
struct X {
    int x, y;

    auto getter() const
    {
        return [*this]<typename Self>() Self&& {
            return x       // still refers to X::x
                + this->y; // still refers to X::y
        };
    }
};
```

§ **3.2.10. Parsing issues**

The object parameter syntaxes (§3.1.1 Explicit this-annotated parameter and §3.1.2 Explicit this parameter) have no parsing issues that we are aware of.

With the addition of a new type name after the *parameter-declaration-clause* (in §3.1.3 Trailing type with identifier and §3.1.4 Trailing type sans identifier), we potentially run into a clash with the existing *virt-specifier*s, especially if allowing for an arbitrary identifier.

Consider:

```
struct B {
    virtual B* override() = 0;
};

struct override : B {
    override* override() override override override; // #1
    override* override() override override;          // #2
    override* override() override;                  // #3
    override* override();                           // #4
};
```

The same problem would occur with `final`.

In order to disambiguate between a trailing object type, a trailing arbitrary identifier, and a *virt-specifier* — or any future trailing context-sensitive keyword — we would have to specify a preference, which would probably be to parse out the type first and the identifier second, assuming an identifier is allowed.

If we go with syntax that allows for an identifier, `#1` will have an explicit object of type `override` named `override` that is an `override`. `#2` would declare an object with an identifier that does not make use of the override *virt-specifier*. `#3` would provide an explicit object type without an identifier. Notably, `#3` is valid code even today, and would remain valid code. Only the meaning of the `override` identifier would change.

If we go with syntax that does *not* allow for an identifier, `#1` would be ill-formed, `#2` would declare an explicit object having type `override` that has the *virt-specifier* `override`, and `#3` would likewise change meaning.

In practice, we do not believe that anybody actually writes code like this, so it is unlikely to break real code.

We feel that allowing for an arbitrary identifier would be grabbing too much real estate with minimal benefit, as it would constrain further evolution of the standard and make it more difficult to use. Let’s say we added a new context-sensitive keyword, like `super`. A user might try to write:

```
struct Y {
    // intending to use the new context-sensitive keyword but
    // really is providing a name to the object parameter?
    void a() super;

    // same
    void b() Y super;

    // okay, these finally use the keyword as desired - the user
    // has to provide an identifier, even if they don't want one
    void c() _ super;
    void d() Y _ super;
};
```

Without an arbitrary identifier, `a()` and `b()` both treat `super` as the context-sensitive keyword, as likely intended. The only edge case would be in a scenario where you have a type named `super`.

Now consider the following:

```
struct Z {
    void foo() cosnt;
};
```

The user made a typo and wrote `cosnt` instead of `const`. In C++17, it's not a problem — this is ill-formed and the compiler will helpfully point out your problem.

But if we allow an arbitrary trailing identifier and try to parse an identifier, even without an explicit type, this suddenly becomes well-formed. We declare a member function with an explicit object type that is implicitly `Z`, and that explicit object is named `cosnt`. Notably, this member function is *not* const!

We may end up needing parsing rules, such as:

1. Try to parse the type.
2. If we find a type, then try to parse an identifier.
3. Only then, consider things like the *virt-specifier*s.

That's complicated, to say the least. It also doesn't quite mesh with the mental model a user might form about what this syntax means — that we have an implicitly-provided object type which is the class type:

```
struct C {
    void foo();   // implicit
    void foo() C; // explicit

    void bar() const&;   // implicit
    void bar() C const&; // explicit

    // so it seems like it should follow that...
    void quux() self;          // implicit
    void quux() C self;        // explicit

    void quuz() const& self;   // implicit
    void quuz() C const& self; // explicit
};
```

Either the `quux` or `quuz` examples fail by making the implicit versions ill-formed, or we end up with this `cosnt` typo problem. Neither option is desirable.

§ 3.3. Comparison of the options

Here is a comparison of the four main syntaxes against a variety of metrics:

	§3.1.1 Explicit this-annotated parameter	§3.1.2 Explicit this parameter	§3.1.3 Trailing type with identifier	§3.1.4 Trailing type sans identifier
Familiarity	Novel to C++, familiar to users of other languages. Adds an additional way to define *cv*- and *ref* qualifiers.	Same unfamiliarity as at-left; `this` behaves consistently with its declaration.	Keeps *cv*- and *ref*-qualifiers where they already are, so it is a natural extension.	
Safety with derived objects†	Easy, if `this` is accessible; verbose otherwise.	Difficult and verbose.	Easy, if `this` is accessible; verbose otherwise.	Verbose.
Parsing issues	None	None	Definite problems requiring a choice of parsing strategy. Limits further extensions with context-sensitive keywords.	Some problems.
Code bloat‡	Cannot deduce `const`/non-`const`, so would usually have to resort to `Self&&`.		Not an issue.	Not an issue.

†*Safety with derived objects*: the simplicity of writing member functions that want to deduce *cv-qualifiers* and value category without inadvertently referencing a shadowing member of the derived object. This table assumes we do not adopt the §3.4 Potential Extension. Syntaxes that do not introduce an identifier (§3.1.2 Explicit this parameter and §3.1.4 Trailing type sans identifier) have no way of providing a non-verbose way of referring to members of the base class, and are therefore marked as "Verbose".

‡Code bloat, in this context, means that we have to determine whether we must produce more instantiations than would be minimally necessary to solve the problem. Deferring to a templated implementation is an acceptable option and has been improved by no longer requiring casts. The problem is minimal.

§ 3.3.1. Use-case analysis

We evaluated how well the various syntax options work for the various use-cases they are meant to solve:

	§3.1.1 Explicit this-annotated parameter	§3.1.2 Explicit this parameter	§3.1.3 Trailing type with identifier	§3.1.4 Trailing type sans identifier
§4.1 Deduplicating Code	Yes	Yes	Yes	Yes
§4.2 CRTP, without the C, R, or even T	Yes	Yes	Yes	Yes
§4.3 Recursive Lambdas	Yes	Yes†	Yes	~~No~~
§4.4 By-value member functions	Yes	Yes	~~No~~	~~No~~
§4.5 SFINAE-friendly callables	Yes	Yes	Yes	Yes

† While both §3.1.1 Explicit this-annotated parameter and §3.1.2 Explicit this parameter support recursive lambdas, nestedly-recursive lambdas are easier if one can assign an identifier in a natural way. §3.1.2 Explicit this parameter also has unwelcome interactions with capturing `this` in the lambda, since the parameter `this` must shadow the captured one.

§ 3.4. Potential Extension

This extension is not explicitly proposed proposed by our paper, since it has not yet been completely explored. Nevertheless, the authors believe that certain concerns raised by the proposed feature may be alleviated by discussing the following possible solution to those issues.

One of the pitfalls of having a deduced object parameter or a deduced trailing object type is when the intent is solely to deduce the *cv*-qualifiers and value category of the object parameter, but a derived type is deduced as well — any access through an object that might have a derived type could inadvertently refer to a shadowed member in the derived class. While this is desirable and very powerful in the case of mixins, it is not always desirable in other situations. Superfluous template instantiations are also unwelcome side effects.

One family of possible solutions could be summarized as **make it easy to get the base class pointer**. However, all of these solutions still require extra instantiations. For `optional::value()`, we really only want four instantiations: `&`, `const&`, `&&`, and `const&&`. If something inherits from `optional`, we don't want additional instantiations of those functions for the derived types, which won't do anything new, anyway. This is code bloat.

C++ already has this long-recognised problem for free function templates. The authors have heard many a complaint about it from library vendors, even before this paper was introduced, as it is desirable to only deduce the ref-qualifier in many contexts. Therefore, it might make sense to tackle this issue in a more general way. A complementary feature could be proposed to constrain *type deduction* as opposed to removing candidates once they are deduced (as accomplished by `requires`), with the following straw-man syntax:

```
struct Base {
    template <typename Self : Base>
    auto front(this Self&& self);
};
struct Derived : Base { };

// also works for free functions
template <typename T : Base>
void foo(T&& x) {
   static_assert(is_same_v<Base, remove_reference_t<T>>);
}

Base{}.front(); // calls Base::front<Base>
Derived{}.front(); // also calls Base::front<Base>

foo(Base{}); // calls foo<Base>
foo(Derived{}); // also calls foo<Base>
```

This would create a function template that only generates functions taking a `Base`, ensuring that we don't generate additional instantiations when those functions participate in overload resolution. Such a proposal would also change how templates participate in overload resolution, however, and is not to be attempted haphazardly.

§ 4. Real-World Examples

What follows are several examples of the kinds of problems that can be solved using this proposal. We provide examples using two different syntaxes, where possible:

- the explicit object parameter syntax, with `this` accessible but implicit lookup disallowed; and
- the explicit object type syntax, with no identifier allowed.

We are not using the straw-man extension syntax to constrain deduction. Instead, we just carefully ensure correctness, where relevant.

§ 4.1. Deduplicating Code

This proposal can de-duplicate and de-quadruplicate a large amount of code. In each case, the single function is only slightly more complex than the initial two or four, which makes for a huge win. What follows are a few examples of ways to reduce repeated code.

This particular implementation of optional is Simon's, and can be viewed on GitHub. It includes some functions proposed in [P0798R0], with minor changes to better suit this format:

C++17	§3.1.1 Explicit this-annotated parameter

```
class TextBlock {
public:
  char const& operator[](size_t position) const {
    // ...
    return text[position];
  }

  char& operator[](size_t position) {
    return const_cast<char&>(
      static_cast<TextBlock const&>
        (this)[position]
    );
  }
  // ...
};
```

```
class TextBlock {
public:
  template <typename Self>
  auto& operator[](this Self&&, size_t position) {
    // ...
    return this->text[position];
  }
  // ...
};
```

```
template <typename T>
class optional {
  // ...
  constexpr T* operator->() {
    return addressof(this->m_value);
  }

  constexpr T const*
  operator->() const {
    return addressof(this->m_value);
  }
  // ...
};
```

```
template <typename T>
class optional {
  // ...
  template <typename Self>
  constexpr auto operator->(this Self&&) {
    return addressof(this->m_value);
  }
  // ...
};
```

```cpp
template <typename T>
class optional {
  // ...
  constexpr T& operator*() & {
    return this->m_value;
  }

  constexpr T const& operator*() const& {
    return this->m_value;
  }

  constexpr T&& operator*() && {
    return move(this->m_value);
  }

  constexpr T const&&
  operator*() const&& {
    return move(this->m_value);
  }

  constexpr T& value() & {
    if (has_value()) {
      return this->m_value;
    }
    throw bad_optional_access();
  }

  constexpr T const& value() const& {
    if (has_value()) {
      return this->m_value;
    }
    throw bad_optional_access();
  }

  constexpr T&& value() && {
    if (has_value()) {
      return move(this->m_value);
    }
    throw bad_optional_access();
  }

  constexpr T const&& value() const&& {
    if (has_value()) {
      return move(this->m_value);
    }
    throw bad_optional_access();
  }
  // ...
};
```

```cpp
template <typename T>
class optional {
  // ...
  template <typename Self>
  constexpr like_t<Self, T>&& operator*(this Self&&
    return forward_like<Self>(*this).m_value;
  }

  template <typename Self>
  constexpr like_t<Self, T>&& value(this Self&&) {
    if (this->has_value()) {
      return forward_like<Self>(*this).m_value);
    }
    throw bad_optional_access();
  }
  // ...
};
```

```
template <typename T>
class optional {
  // ...
  template <typename F>
  constexpr auto and_then(F&& f) & {
    using result =
      invoke_result_t<F, T&>;
    static_assert(
      is_optional<result>::value,
      "F must return an optional");

    return has_value()
        ? invoke(forward<F>(f), **this)
        : nullopt;
  }

  template <typename F>
  constexpr auto and_then(F&& f) && {
    using result =
      invoke_result_t<F, T&&>;
    static_assert(
      is_optional<result>::value,
      "F must return an optional");

    return has_value()
        ? invoke(forward<F>(f),
                 move(**this))
        : nullopt;
  }

  template <typename F>
  constexpr auto and_then(F&& f) const& {
    using result =
      invoke_result_t<F, T const&>;
    static_assert(
      is_optional<result>::value,
      "F must return an optional");

    return has_value()
        ? invoke(forward<F>(f), **this)
        : nullopt;
  }

  template <typename F>
  constexpr auto and_then(F&& f) const&& {
    using result =
      invoke_result_t<F, T const&&>;
    static_assert(
      is_optional<result>::value,
      "F must return an optional");

    return has_value()
        ? invoke(forward<F>(f),
                 move(**this))
        : nullopt;
  }
  // ...
};
```

```
template <typename T>
class optional {
  // ...
  template <typename Self, typename F>
  constexpr auto and_then(this Self&&, F&& f) {
    using val = decltype((
        forward_like<Self>(*this).m_value));
    using result = invoke_result_t<F, val>;

    static_assert(
      is_optional<result>::value,
      "F must return an optional");

    return this->has_value()
        ? invoke(forward<F>(f),
                 forward_like<Self>(*this).m_value)
        : nullopt;
  }
  // ...
};
```

There are a few more functions in P0798 responsible for this explosion of overloads, so the difference in both code and clarity is dramatic.

For those that dislike returning auto in these cases, it is easy to write a metafunction matching the appropriate qualifiers from a type. It is certainly a better option than blindly copying and pasting code, hoping that the minor changes were made correctly in each case.

§ 4.2. CRTP, without the C, R, or even T

Today, a common design pattern is the Curiously Recurring Template Pattern. This implies passing the derived type as a template parameter to a base class template as a way of achieving static polymorphism. If we wanted to simply outsource implementing postfix incrementation to a base, we could use CRTP for that. But with explicit objects that already deduce to the derived objects, we don't need any curious recurrence — we can use standard inheritance and let deduction do its thing. The base class doesn't even need to be a template:

C++17	§3.1.1 Explicit this-annotated parameter

`template <typename Derived>`
`struct add_postfix_increment {`
`  Derived operator++(int) {`
`  auto& self = static_cast<Derived&>(*this);`
 	
`  Derived tmp(self);`
`  ++self;`
`  return tmp;`
`  }`
`};`
 	
`struct some_type : add_postfix_increment<some_type> {`
`  some_type& operator++() { ... }`
`};`
	`struct add_postfix_increment {`
	`  template <typename Self>`
	`  auto operator++(this Self&& self, int) {`
	`  auto tmp = self;`
	`  ++self;`
	`  return tmp;`
	`  }`
	`};`
	
	`struct some_type : add_postfix_increment {`
	`  some_type& operator++() { ... }`
	`};`

The proposed examples aren't much shorter, but they are certainly simpler by comparison.

§ 4.2.1. Builder pattern

Once we start to do any more with CRTP, complexity quickly increases, whereas with this proposal, it stays remarkably low.

Let's say we have a builder that does multiple things. We might start with:

```
struct Builder {
  Builder& a() { /* ... */; return *this; }
  Builder& b() { /* ... */; return *this; }
  Builder& c() { /* ... */; return *this; }
};

Builder().a().b().a().b().c();
```

But now we want to create a specialized builder with new operations `d()` and `e()`. This specialized builder needs new member functions, and we don't want to burden existing users with them. We also want `Special().a().d()` to work, so we need to use CRTP to *conditionally* return either a `Builder&` or a `Special&`:

C++17	§3.1.1 Explicit this-annotated parame

```cpp
template <typename D=void>
class Builder {
  using Derived = conditional_t<is_void_v<D>, Builder, D>;
  Derived& self() {
    return *static_cast<Derived*>(this);
  }

public:
  Derived& a() { /* ... */; return self(); }
  Derived& b() { /* ... */; return self(); }
  Derived& c() { /* ... */; return self(); }
};

struct Special : Builder<Special> {
  Special& d() { /* ... */; return *this; }
  Special& e() { /* ... */; return *this; }
};

Builder().a().b().a().b().c();
Special().a().d().e().a();
```

```cpp
struct Builder {
    template <typename Self>
    Self& a(this Self&& self) { /* ... */;

    template <typename Self>
    Self& b(this Self&& self) { /* ... */;

    template <typename Self>
    Self& c(this Self&& self) { /* ... */;
};

struct Special : Builder {
    Special& d() { /* ... */; return *this
    Special& e() { /* ... */; return *this
};

Builder().a().b().a().b().c();
Special().a().d().e().a();
```

The code on the right is dramatically easier to understand and therefore more accessible to more programmers than the code on the left.

But wait! There's more!

What if we added a *super*-specialized builder, a more special form of `Special`? Now we need `Special` to opt-in to CRTP so that it knows which type to pass to `Builder`, ensuring that everything in the hierarchy returns the correct type. It's about this point that most programmers would give up. But with this proposal, there's no problem!

C++17	§3.1.1 Explicit this-annotated pa

```cpp
template <typename D=void>
class Builder {
protected:
  using Derived = conditional_t<is_void_v<D>, Builder, D>;
  Derived& self() {
    return *static_cast<Derived*>(this);
  }

public:
  Derived& a() { /* ... */; return self(); }
  Derived& b() { /* ... */; return self(); }
  Derived& c() { /* ... */; return self(); }
};

template <typename D=void>
struct Special
  : Builder<conditional_t<is_void_v<D>,Special<D>,D>
{
  using Derived = typename Special::Builder::Derived;
  Derived& d() { /* ... */; return this->self(); }
  Derived& e() { /* ... */; return this->self(); }
};

struct Super : Special<Super>
{
    Super& f() { /* ... */; return *this; }
};

Builder().a().b().a().b().c();
Special().a().d().e().a();
Super().a().d().f().e();
```

```cpp
struct Builder {
    template <typename Self>
    Self& a(this Self&& self) { /* ... */;

    template <typename Self>
    Self& b(this Self&& self) { /* ... */;

    template <typename Self>
    Self& c(this Self&& self) { /* ... */;
};

struct Special : Builder {
    template <typename Self>
    Self& d(this Self&& self) { /* ... */;

    template <typename Self>
    Self& e(this Self&& self) { /* ... */;
};

struct Super : Special {
    template <typename Self>
    Self& f(this Self&& self) { /* ... */;
};

Builder().a().b().a().b().c();
Special().a().d().e().a();
Super().a().d().f().e();
```

The code on the right is much easier in all contexts. There are so many situations where this idiom, if available, would give programmers a better solution for problems that they cannot easily solve today.

Note that the `Super` implementations with this proposal opt-in to further derivation, since it's a no-brainer at this point.

§ 4.3. Recursive Lambdas

The explicit object parameter syntax offers an alternative solution to implementing a recursive lambda as compared to [P0839R0], since now we've opened up the possibility of allowing a lambda to reference itself. To do this, we need a way to *name* the lambda. The trailing object type syntax does not allow for an identifier, and therefore *does not* help us solve this problem.

```cpp
// as proposed in P0839
auto fib = [] self (int n) {
    if (n < 2) return n;
    return self(n-1) + self(n-2);
};

// this proposal
auto fib = [](this auto const& self, int n) {
    if (n < 2) return n;
    return self(n-1) + self(n-2);
};
```

This works by following the established rules. The call operator of the closure object can also have an explicit object parameter, so in this example, `self` is the closure object.

Combine this with the new style of mixins allowing us to automatically deduce the most derived object, and you get the following example — a simple recursive lambda that counts the number of leaves in a tree.

```
struct Node;
using Tree = variant<Leaf, Node*>;
struct Node {
    Tree left;
    Tree right;
};

int num_leaves(Tree const& tree) {
    return visit(overload(        // <-----------------------------------+
        [](Leaf const&) { return 1; },                                  //      |
        [](this auto const& self, Node* n) -> int {                     //      |
            return visit(self, n->left) + visit(self, n->right); // <----+
        }
    ), tree);
}
```

In the calls to `visit`, `self` isn't the lambda; `self` is the `overload` wrapper. This works straight out of the box.

§ 4.4. By-value member functions

This section presents some of the cases for by-value member functions.

§ 4.4.1. For move-into-parameter chaining

Say you wanted to provide a `.sorted()` method on a data structure. Such a method naturally wants to operate on a copy. Taking the parameter by value will cleanly and correctly move into the parameter if the original object is an rvalue without requiring templates.

```
struct my_vector : vector<int> {
  auto sorted(this my_vector self) -> my_vector {
    sort(self.begin(), self.end());
    return self;
  }
};
```

§ 4.4.2. For performance

It's been established that if you want the best performance, you should pass small types by value to avoid an indirection penalty. One such small type is `std::string_view`. Abseil Tip #1 for instance, states:

> Unlike other string types, you should pass `string_view` by value just like you would an `int` or a `double` because `string_view` is a small value.

There is, however, one place today where you simply *cannot* pass types like `string_view` by value: to their own member functions. The implicit object parameter is always a reference, so any such member functions that do not get inlined incur a double indirection.

As an easy performance optimization, any member function of small types that does not perform any modifications can take the object parameter by value. This is possible *only* when using the explicit object parameter syntax; there is no way to express this idea with a trailing type. Here is an example of some member functions of `basic_string_view` assuming that we are just using `charT const*` as `iterator`:

```
template <class charT, class traits = char_traits<charT>>
class basic_string_view {
private:
    const_pointer data_;
    size_type size_;
public:
    constexpr const_iterator begin(this basic_string_view self) {
        return self.data_;
    }

    constexpr const_iterator end(this basic_string_view self) {
        return self.data_ + self.size_;
    }

    constexpr size_t size(this basic_string_view self) {
        return self.size_;
    }

    constexpr const_reference operator[](this basic_string_view self, size_type pos) {
        return self.data_[pos];
    }
};
```

Most of the member functions can be rewritten this way for a free performance boost.

The same can be said for types that aren't only cheap to copy, but have no state at all. Compare these two implementations of `less_than`:

C++17	§3.1.1 Explicit this-annotated parameter

`struct less_than {`
`  template <typename T, typename U>`
`  bool operator()(T const& lhs, U const& rhs) {`
`  return lhs < rhs;`
`  }`
`};`
	`struct less_than {`
	`  template <typename T, typename U>`
	`  bool operator()(this less_than,`
	`  T const& lhs, U const& rhs) {`
	`  return lhs < rhs;`
	`  }`
	`};`

In C++17, invoking `less_than()(x, y)` still requires an implicit reference to the `less_than` object — completely unnecessary work when copying it is free. The compiler knows it doesn't have to do anything. We *want* to pass `less_than` by value here. Indeed, this specific situation is the main motivation for [P1169R0].

§ 4.5. SFINAE-friendly callables

A seemingly unrelated problem to the question of code quadruplication is that of writing numerous overloads for function wrappers, as demonstrated in [P0826R0]. Consider what happens if we implement `std::not_fn()` as currently specified:

```
template <typename F>
class call_wrapper {
    F f;
public:
    // ...
    template <typename... Args>
    auto operator()(Args&&... ) &
        -> decltype(!declval<invoke_result_t<F&, Args...>>());

    template <typename... Args>
    auto operator()(Args&&... ) const&
        -> decltype(!declval<invoke_result_t<F const&, Args...>>());

    // ... same for && and const && ...
};

template <typename F>
auto not_fn(F&& f) {
    return call_wrapper<decay_t<F>>{forward<F>(f)};
}
```

As described in the paper, this implementation has two pathological cases: one in which the callable is SFINAE-unfriendly, causing the call to be ill-formed where it would otherwise work; and one in which overload is deleted, causing the call to fall back to a different overload when it should fail instead:

```
struct unfriendly {
    template <typename T>
    auto operator()(T v) {
        static_assert(is_same_v<T, int>);
        return v;
    }

    template <typename T>
    auto operator()(T v) const {
        static_assert(is_same_v<T, double>);
        return v;
    }
};

struct fun {
    template <typename... Args>
    void operator()(Args&&...) = delete;

    template <typename... Args>
    bool operator()(Args&&...) const { return true; }
};

std::not_fn(unfriendly{})(1); // static assert!
                              // even though the non-const overload is viable and would be
                              // best match, during overload resolution, both overloads of
                              // unfriendly have to be instantiated - and the second one is
                              // hard compile error.

std::not_fn(fun{})();         // ok!? Returns false
                              // even though we want the non-const overload to be deleted,
                              // const overload of the call_wrapper ends up being viable -
                              // the only viable candidate.
```

Gracefully handling SFINAE-unfriendly callables is **not solvable** in C++ today. Preventing fallback can be solved by the addition of another four overloads, so that each of the four *cv*/ref-qualifiers leads to a pair of overloads: one enabled and one `deleted`.

This proposal solves both problems by allowing `this` to be deduced. The following is a complete implementation of `std::not_fn`. For simplicity, it makes use of `BOOST_HOF_RETURNS` from Boost.HOF to avoid duplicating expressions:

§3.1.1 Explicit this-annotated parameter

```
template <typename F>
struct call_wrapper {
  F f;

  template <typename Self, typename... Args>
  auto operator()(this Self&&, Args&&... args)
    BOOST_HOF_RETURNS(
      !invoke(
        forward_like<Self>(this->f),
        forward<Args>(args)...))
};

template <typename F>
auto not_fn(F&& f) {
  return call_wrapper<decay_t<F>>{forward<F>(f)};
}
```

§3.1.4 Trailing type sans identifier

```
template <typename F>
struct call_wrapper {
  F f;

  template <typename Self, typename... Args>
  auto operator()(Args&&... args) Self&&
    BOOST_HOF_RETURNS(
      !invoke(
        forward_like<Self>(this->call_wrapper::f),
        forward<Args>(args)...))
};

template <typename F>
auto not_fn(F&& f) {
  return call_wrapper<decay_t<F>>{forward<F>(f)};
}
```

With either syntax:

```
not_fn(unfriendly{})(1); // ok
not_fn(fun{})();         // error
```

Here, there is only one overload with everything deduced together. The first example now works correctly. `Self` gets deduced as `call_wrapper<unfriendly>`, and the one `operator()` will only consider `unfriendly`'s non-`const` call operator. The `const` one is never even considered, so it does not have an opportunity to cause problems.

The second example now also fails correctly. Previously, we had four candidates. The two non-`const` options were removed from the overload set due to `fun`'s non-`const` call operator being `delete`d, and the two `const` ones which were viable. But now, we only have one candidate. `Self` is deduced as `call_wrapper<fun>`, which requires `fun`'s non-`const` call operator to be well-formed. Since it is not, the call results in an error. There is no opportunity for fallback since only one overload is ever considered.

This singular overload has precisely the desired behavior: working for `unfriendly`, and not working for `fun`.

This could also be implemented as a lambda completely within the body of `not_fn`:

```
template <typename F>
auto not_fn(F&& f) {
    return [f=forward<F>(f)](this auto&& self, auto&&.. args)
        BOOST_HOF_RETURNS(
            !invoke(
                forward_like<decltype(self)>(f),
                forward<decltype(args)>(args)...))
        ;
}
```

§ 5. Suggested Polls

We would like to suggest the following polls on this proposal:

1. Do we want a solution in the direction of this paper — that is, do we want a syntax that allows for deducing the qualifiers and value category of the object parameter for non-static member functions?
2. [Four-way] We want that solution to be
    - §3.1.1 Explicit this-annotated parameter

- §3.1.2 Explicit this parameter
- §3.1.3 Trailing type with identifier
- §3.1.4 Trailing type sans identifier

3. [three-way] Only if §3.1.1 Explicit this-annotated parameter or §3.1.3 Trailing type with identifier is chosen:
   - `this` should be inaccessible and every access must be qualified
   - `this` is accessible but every access must be qualified
   - `this` is accessible and implicit lookup is permitted.
4. Should we pursue a solution along the lines presented §3.4 Potential Extension `typename T: B`?

§ 6. Acknowledgements

The authors would like to thank:

- Jonathan Wakely, for bringing us all together by pointing out we were writing the same paper, twice
- Chandler Carruth for a lot of feedback and guidance around many design issues, but especially for help with use cases and the pointer-types for by-value passing
- Graham Heynes, Andrew Bennieston, Jeff Snyder for early feedback regarding the meaning of `this` inside function bodies
- Amy Worthington, Jackie Chen, Vittorio Romeo, Tristan Brindle, Agustín Bergé, Louis Dionne, and Michael Park for early feedback
- Guilherme Hartmann for his guidance with the implementation
- Richard Smith, Jens Maurer, and Hubert Tong for help with wording
- Ville Voutilainen, Herb Sutter, Titus Winters and Bjarne Stroustrup for their guidance in design-space exploration
- Eva Conti for furious copy editing, patience, and moral support

§ References

§ Informative References

[Effective]
Scott Meyers. Effective C++, Third Edition. 2005. URL: https://www.aristeia.com/books.html

[P0798R0]
Simon Brand. Monadic operations for std::optional. 6 October 2017. URL: https://wg21.link/p0798r0

[P0826R0]
Agustín Bergé. SFINAE-friendly std::bind. 12 October 2017. URL: https://wg21.link/p0826r0

[P0839R0]
Richard Smith. Recursive Lambdas. 16 October 2017. URL: https://wg21.link/p0839r0

[P0847R0]
Gašper Ažman, Simon Brand, Ben Deane, Barry Revzin. Deducing this. 12 February 2018. URL: https://wg21.link/p0847r0

[P0929R2]
Jens Maurer. Checking for abstract class types. 6 June 2018. URL: https://wg21.link/p0929r2

[P1169R0]
Barry Revzin; Casey Carter. static operator(). 7 October 2018. URL: https://wg21.link/p1169r0